

Spectrum similarity for Bruker NMR

This tool implements the bin-based spectrum similarity measure described in:

Lorant Bodis, Alfred Ross, Erno Pretsch, "A novel spectra similarity measure", Chemometrics and Intelligent Laboratory Systems 85 (2007) 1-8.

`spectrum_similarity.py` handles 1D spectra; `hsqc_similarity.py` extends the same method to 2D (HSQC and other processed 2D experiments).

Method

Two spectra x and y are compared inside a common chemical-shift window. Each processed point p has a chemical shift $\delta(p)$ and an intensity $I_a(p)$ ($a \in \{x, y\}$). With the default `clip` baseline, negative intensities are zeroed first, $I_a(p) \leftarrow \max(I_a(p), 0)$.

Every point carries a weight $w(p)$ equal to its local integration element, so spectra of different digital resolution integrate consistently:

$$w_{1D}(p) = |\Delta\delta(p)|, \quad w_{2D}(p) = |\Delta\delta_{F1}(p)| \cdot |\Delta\delta_{F2}(p)|.$$

Binning. At resolution n the window is split into equal bins. In 1D, a window $[\delta_{lo}, \delta_{hi}]$ of width $W = \delta_{hi} - \delta_{lo}$ gives n bins with edges $e_k = \delta_{lo} + kW / n$ ($k = 0, \dots, n$). In 2D the same is done independently along $F2$ and $F1$, producing an $n \times n$ grid. The raw integral of bin k and its normalization to a target integral N_a (`--norm-x / --norm-y`, default 1):

$$A_k^a(n) = \sum_{p: \delta(p) \in \text{bin } k} I_a(p) w(p), \quad b_k^a(n) = N_a \frac{A_k^a(n)}{\sum_{k'} A_{k'}^a(n)}, \quad \sum_k b_k^a(n) = N_a.$$

Similarity index. Each resolution yields a weighted Jaccard (Ruzicka) index, clamped to $[0, 1]$:

$$SI_n = \frac{\sum_k \min(b_k^x, b_k^y)}{N_x + N_y - \sum_k \min(b_k^x, b_k^y)} = \frac{\sum_k \min(b_k^x, b_k^y)}{\sum_k \max(b_k^x, b_k^y)},$$

using $\sum_k \min + \sum_k \max = N_x + N_y$. In 2D the sums run over grid bins (i, j) with $b_{ij}^a(n)$ replacing $b_k^a(n)$.

The resolution runs $n = 1, \dots, N_{\text{bins}}$, where the finest bins are bounded by the minimum bin width $w_{\min}$:

$$N_{\text{bins}}^{1\text{D}} = \left\lfloor \frac{W}{w_{\min}} \right\rfloor, \quad N_{\text{bins}}^{2\text{D}} = \min \left(\left\lfloor \frac{W_{F2}}{w_{\min, F2}} \right\rfloor, \left\lfloor \frac{W_{F1}}{w_{\min, F1}} \right\rfloor \right).$$

Upper envelope. At $n = 1$ every pair of normalized spectra collapses to a single bin, so $\text{SI}_1 = 1$ is uninformative; the envelope SI_n^* smooths the descending SI_n curve. Anchors are chosen greedily: set $\text{SI}_1^* = 1$; given an anchor at position m , the next anchor is the largest index $j > m$ that attains $\max_{k > m} \text{SI}_k$. Between consecutive anchors SI_n^* is linear in n , and finally $\text{SI}_n^* = \max(\text{interpolation}, \text{SI}_n)$ so that $\text{SI}_n^* \geq \text{SI}_n$ everywhere.

Score. The reported similarity is the mean of the envelope over all resolutions:

$$S = \frac{1}{N_{\text{bins}}} \sum_{n=1}^{N_{\text{bins}}} \text{SI}_n^* \in [0, 1].$$

Identical spectra give $\text{SI}_n = 1$ for every n , hence $S = 1$. Because S depends on N_{bins} , only compare scores computed with the same minimum bin width.

1D

The input is a processed Bruker 1D spectrum. Pass either an experiment directory, a `pdata` directory, or a processed `1r` file. The program reads `procs` and `1r`.

Usage

```
python3 spectrum_similarity.py /path/to/exp1 /path/to/exp2
```

Useful options:

```
python3 spectrum_similarity.py exp1 exp2 --procno 1 --ppm-min 0 --ppm-max 10
python3 spectrum_similarity.py exp1 exp2 --min-bin-width 0.4 --json
```

```
python3 spectrum_similarity.py exp1 exp2 --norm-x 19 --norm-y 19
python3 spectrum_similarity.py exp1 exp2 --plot result.png
python3 spectrum_similarity.py exp1 exp2 --plot --show
```

Defaults:

- `--min-bin-width 0.4` : the value recommended in the paper for ^1H NMR.
- `--baseline clip` : negative intensities are set to zero before integration.
- If no ppm range is supplied, the common ppm overlap of the two spectra is used.
- Each spectrum is normalized to total integral 1 unless `--norm-x` and `--norm-y` are supplied. For predicted or assigned proton spectra, set these to the proton counts when desired.
- `--plot` saves a graph with the two normalized spectra and the `SIn / SI*n` similarity curves. Without a path it writes `spectrum_similarity.png`.
- `--show` displays the same graph interactively when a GUI backend is available.

2D HSQC

`hsqc_similarity.py` applies the same bin-method concept to processed Bruker 2D spectra (HSQC, HMBC, COSY, ...). Each spectrum is subdivided into an `n` by `n` grid of bins whose widths shrink as `n` grows, and the per-grid `SIn` index and its `SI*n` envelope are averaged into one score in `[0, 1]`.

```
python3 hsqc_similarity.py /path/to/exp1 /path/to/exp2
python3 hsqc_similarity.py exp1 exp2 --f2-min 0 --f2-max 10 --f1-min 0
--f1-max 160
python3 hsqc_similarity.py exp1 exp2 --min-bin-width-f2 0.1 --min-bin-
width-f1 1.0
python3 hsqc_similarity.py exp1 exp2 --plot result.png
python3 hsqc_similarity.py exp1 exp2 --json
```

The reader targets processed 2D Bruker data and reconstructs the submatrix (tile) layout:

```

experiment/
  pdata/
    1/
      2rr
      procs      # direct dimension F2 (e.g. 1H)
      proc2s     # indirect dimension F1 (e.g. 13C or 15N)

```

It reads `SI` , `OFFSET` , `SW_p` , `SF` , and `XDIM` from `procs` / `proc2s` , plus `BYTORDP` , `DTYPP` , and `NC_proc` from `procs` , to decode `2rr` and build both ppm axes.

Defaults:

- `--min-bin-width-f2 0.1` : near the 1H peak linewidth, fine enough to resolve small chemical-shift changes without dropping below the digital resolution.
- `--min-bin-width-f1 1.0` : the matching value for the 13C/15N axis, whose ppm range is roughly ten times larger. Reduce both together for more discrimination between clearly different spectra; the absolute score drops as bins shrink, so only compare scores taken at the same bin widths.
- If no ppm range is supplied for a dimension, the common overlap of the two spectra is used in that dimension.

Bruker assumptions

The reader targets processed 1D Bruker data:

```

experiment/
  pdata/
    1/
      1r
      procs

```

It uses `SI` , `OFFSET` , `SW_p` , `SF` , `BYTORDP` , `DTYPP` , and `NC_proc` from `procs` to decode the data and build the ppm axis.

Test

```
python3 -m pytest
```